



WD2100 Front End Frameworks – Week 3



Tailwind

Atlas
SCHOOL



Tailwind

Tailwind is a utility class framework

A style architecture that favors small, single-purpose classes with names based on visual function.

```
.bg-blue { background-color: #357edd; }  
.text-sm { font-size: 0.875rem; }  
.text-lg { font-size: 1.125rem; }  
.my-0 { margin-top: 0; margin-bottom: 0; }  
.mx-4 { margin-left: 1rem; margin-right: 1rem; }
```

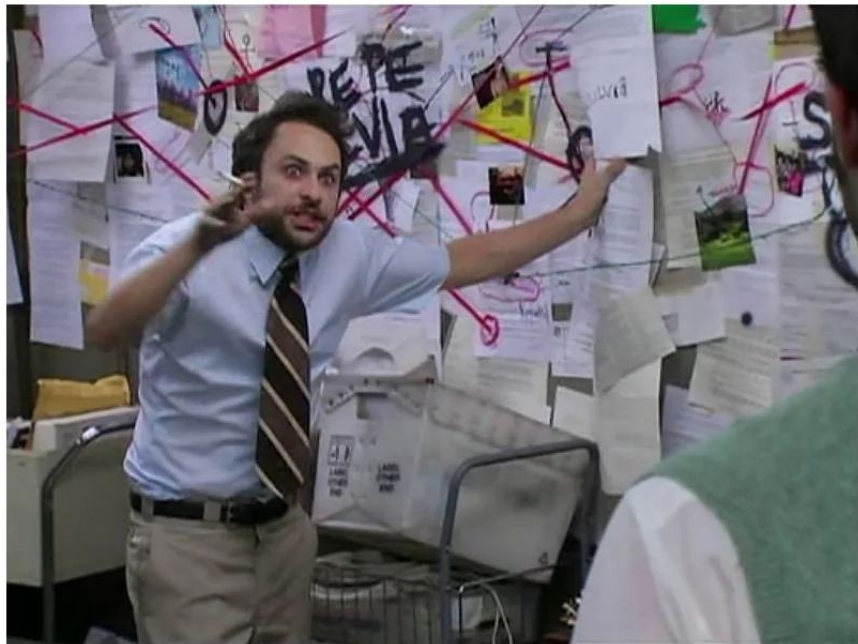
```
function Heading(){  
  return (  
    <h1 className="text-lg mx-4 bg-blue">  
      Hello World  
    </h1>  
  )  
}
```

Tailwind

Naming things is hard

“There are 2 hard problems in computer science: cache invalidation, naming things, and off-by-1 errors.”

Me explaining my variable naming scheme to the other devs



Tailwind

CSS specificity is hard

Selector types:

- ID Selectors
- Class selectors
- Type Selectors

```
button {  
  background-color: red;  
}  
  
#my-button {  
  background-color: blue;  
}  
  
.primary-button {  
  background-color: green;  
}
```

```
<button id="my-button" class="primary-button">
```

Tailwind



<https://johnpolacek.medium.com/by-the-numbers-a-year-and-half-with-atomic-css-39d75b1263b4>

Tailwind

The Paradigm Shift

Traditional: Separate files, inventing class names
(`.chat-notification-wrapper`), context switching.

Tailwind: Composing designs directly in markup using
pre-defined utility classes.

Benefit: No dead CSS, faster prototyping, and instant
consistency with your design system.

The Just-in-Time (JIT) Engine

On-Demand CSS

Tailwind doesn't ship a massive CSS file. It scans your HTML/JSX/TSX files for class names.

It generates only the CSS you actually use, resulting in tiny production builds (often <10kb).

This enables arbitrary values like `w-[17px]` or `bg-[#bada55]` without configuration.



Anatomy of a Utility Class

State Variants

```
hover:bg-blue-600
```

Apply styles only when specific states are active. Works for `focus:`, `active:`, `disabled:`, and even `group-hover:`.

Responsive Prefixes

```
md:w-1/2
```

The style applies only at the md (768px) breakpoint and above. This is the core of responsive design.

Colors & Backgrounds

The Default Palette

- **Scale:** Colors range from 50 (lightest) to 950 (darkest).
- **Backgrounds:** bg-blue-500, bg-zinc-950, bg-white.
- **Text:** text-slate-800, text-red-600, text-transparent.
- **Opacity:** Modify transparency easily: bg-black/50 or text-white/80.



The Spacing Scale



The 4px Grid Rule

→ **The Golden Rule:** 1 unit = 0.25rem = 4px.

→ p-1 = 4px padding.

→ m-4 = 16px margin.

→ w-16 = 64px width.

→ **Padding:** p-4 (all sides), px-4 (horizontal), py-2 (vertical).

→ **Negative Margins:** -m-4 pulls the element closer.

Sizing System

- Tailwind uses **rems** for its sizing.
- Used by:
 - Padding
 - Margin
 - Text Size
 - Height
 - Width
 - Many More...

```
.p-1    { padding: 0.25rem; }  
.p-2    { padding: 0.5rem;  }  
.p-3    { padding: 0.75rem; }  
.p-4    { padding: 1rem;   }  
.p-5    { padding: 1.25rem; }  
.p-6    { padding: 1.5rem;  }  
.p-7    { padding: 1.75rem; }  
.p-8    { padding: 2rem;   }
```

Sizing System

- **em** a unit of measurement that represents the font size of the current element
- **rem** (root em) a unit of measurement that represents the font size of the root element.
- 1rem equals the font size of the html element
- most browsers has a default value of 16px.
- rem ensures web accessibility by allowing users to adjust font sizes to their preference.
- Why you should never set a root font size:
<https://adrianroselli.com/2024/03/the-ultimate-ideal-bestest-base-font-size-that-everyone-is-keeping-a-secret-especially-chet.html>

Sizing System

- There is not a utility class for every possible number
- As the numbers increase the space between utility classes increases

```
.p-60 { padding: 15rem; }  
.p-64 { padding: 16rem; }  
.p-72 { padding: 18rem; }  
.p-80 { padding: 20rem; }  
.p-96 { padding: 24rem; }
```

Typography Essentials



Size & Weight

Classes like text-sm (14px), text-xl (20px), and text-4xl (36px). Weights range from font-thin to font-black.



Alignment & Spacing

text-center, text-right. Control line height with leading-tight or letter spacing with tracking-wide.



Styling

italic, underline, uppercase, capitalize. Truncate long text with a single class: truncate.

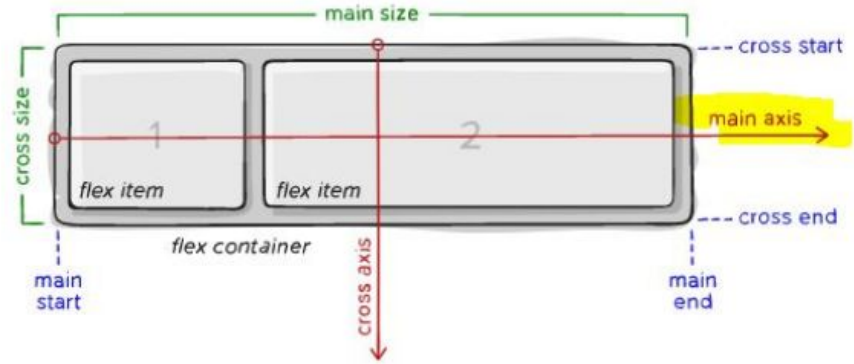
Flexbox & Grid Utilities

Layout Made Simple

Flexbox: flex items-center justify-between handles 90% of UI components (navbars, cards).

Grid: grid grid-cols-3 gap-4 creates a responsive 3-column layout instantly.

No more writing complex float hacks or calculating percentages manually.



Borders & Effects



Borders

`border (1px)`, `border-2`,
`border-t-4`. Colors:
`border-zinc-200`. Styles:
`border-dashed`.



Radius

`rounded-sm`, `rounded-lg`,
`rounded-full` (circles/pills). You
can round specific corners:
`rounded-t-lg`.

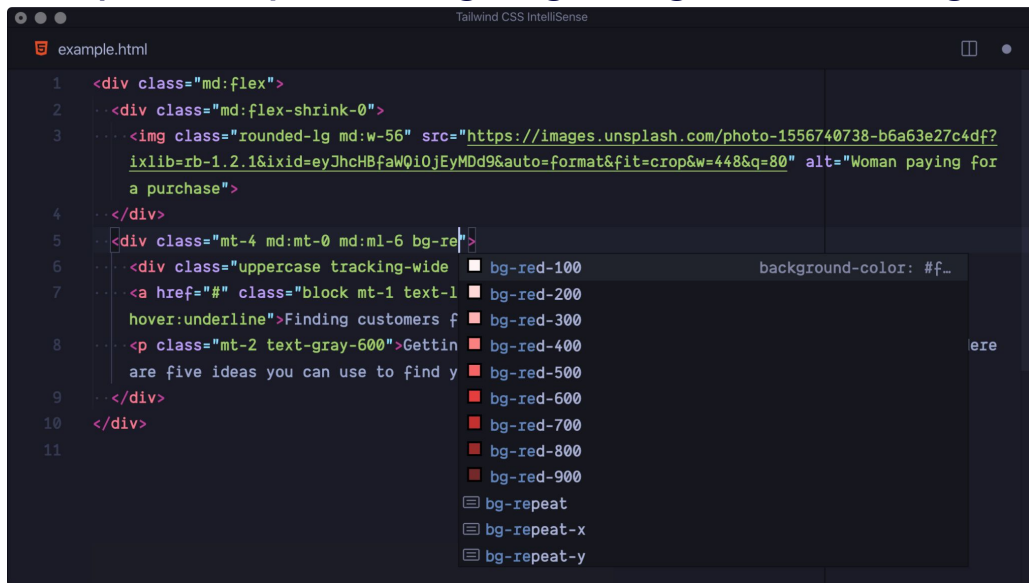


Shadows

`shadow-sm`, `shadow-lg`,
`shadow-xl`. Use `shadow-inner`
for inset depth. No explicit CSS
`box-shadow` needed.

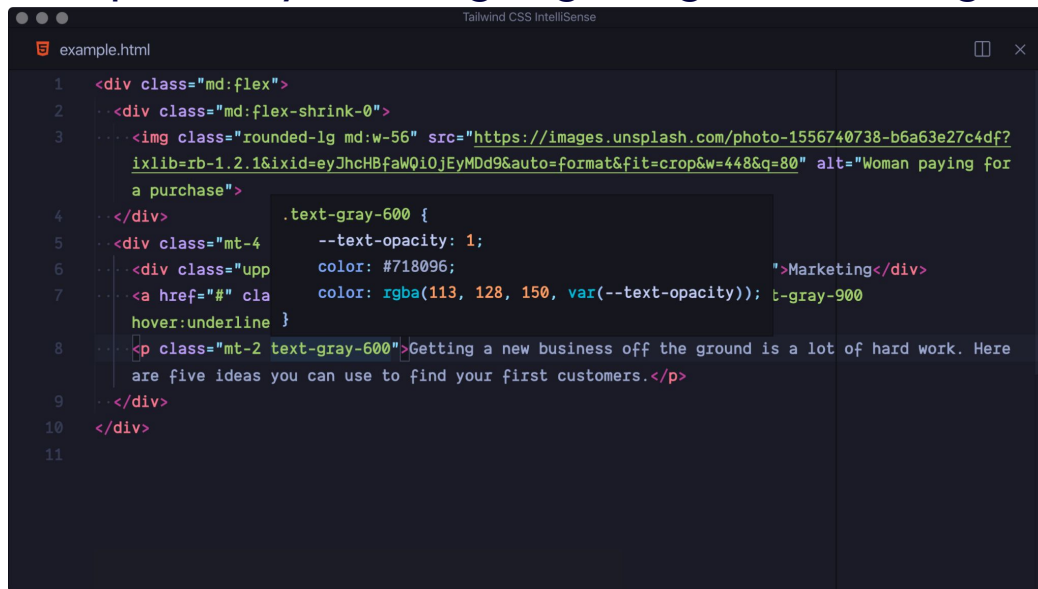
Tools – Tailwind CSS IntelliSense

Visual Studio Code plugin that provides users with advanced features such as autocomplete, syntax highlighting, and linting.



Tools – Tailwind CSS IntelliSense

Visual Studio Code plugin that provides users with advanced features such as autocomplete, syntax highlighting, and linting.



```
1 <div class="md:flex">
2   <div class="md:flex-shrink-0">
3     
4   </div>
5   <div class="mt-4">
6     <div class="upp">
7       <a href="#" class="text-gray-600">Marketing</div>
8     <p class="mt-2 text-gray-600">Getting a new business off the ground is a lot of hard work. Here are five ideas you can use to find your first customers.</p>
9   </div>
10 </div>
11
```

Tools – Prettier Automatic Class Sorting

A plugin that sorts your tailwind classes automatically following the **recommended class order**.

```
<!-- Before -->
<button class="text-white px-4 sm:px-8 py-2 sm:py-3 bg-sky-700 hover:bg-sky-800">

<!-- After -->
<button class="bg-sky-700 px-4 py-2 text-white hover:bg-sky-800 sm:px-8 sm:py-3">
```

<https://github.com/tailwindlabs/prettier-plugin-tailwindcss>

Tools – Prettier Automatic Class Sorting

CSS classes are applied base on their order in the style sheet, not their order in the class attribute

```
.bg-red { background-color: #ff0000; }  
.bg-blue { background-color: #0000ff; }
```

```
<button class="bg-red bg-blue">
```

```
<button class="bg-blue bg-red">
```

Tools – Prettier Automatic Class Sorting

CSS classes are applied base on their order in the styles sheet, not their order in the class attribute

```
.bg-red { background-color: #ff0000; }  
.bg-blue { background-color: #0000ff; }
```

```
<button class="bg-red bg-blue">
```

```
<button class="bg-blue bg-red">
```

Both buttons are blue

Customizing the Tailwind Design System

- Any part of the tailwind design system can be customized
- Other than custom colors and fonts you probably shouldn't customize the design system unless you are an experienced designer.
- The Tailwind defaults should be good for most use cases

```
/** @type {import('tailwindcss').Config} */
module.exports = {
  content: ['./src/**/*.html', './src/**/*.js'],
  theme: {
    colors: {
      'blue': '#1fb6ff',
      'purple': '#7e5bef',
      'pink': '#ff49db',
      'orange': '#ff7849',
      'green': '#13ce66',
      'yellow': '#ffc82c',
      'gray-dark': '#273444',
      'gray': '#8492a6',
      'gray-light': '#d3dce6',
    },
    fontFamily: {
      sans: ['Graphik', 'sans-serif'],
      serif: ['Merriweather', 'serif'],
    },
    extend: {
      spacing: {
        '8xl': '96rem',
        '9xl': '128rem',
      },
      borderRadius: {
        '4xl': '2rem',
      }
    }
  }
},
}
```

Customizing the Tailwind Design System

- You can use square brackets to define custom utility classes inline
- This is not recommended
- Ok for one offs but consider adding the custom utility class to the tailwind config

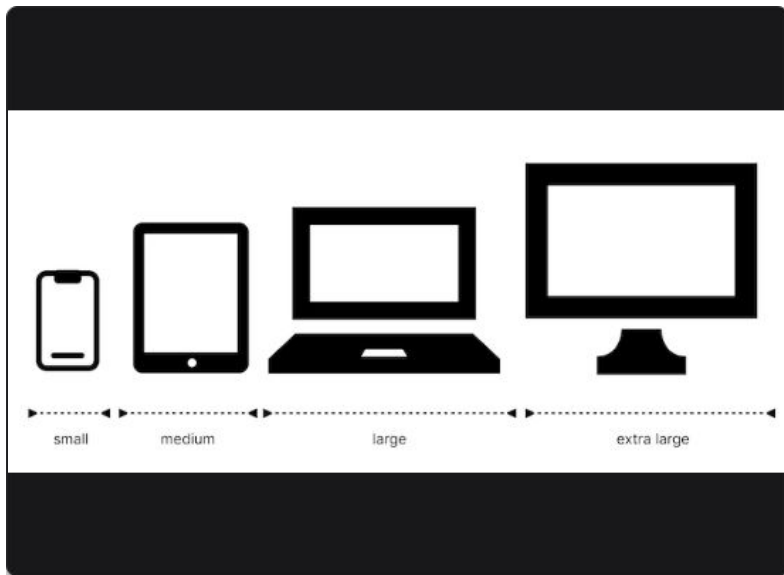
```
<button class="p-[32px] bg-[#928371]">
```

```
<button class="p-[32%]">
```


Responsive Design



Understanding Breakpoints



Standard Config

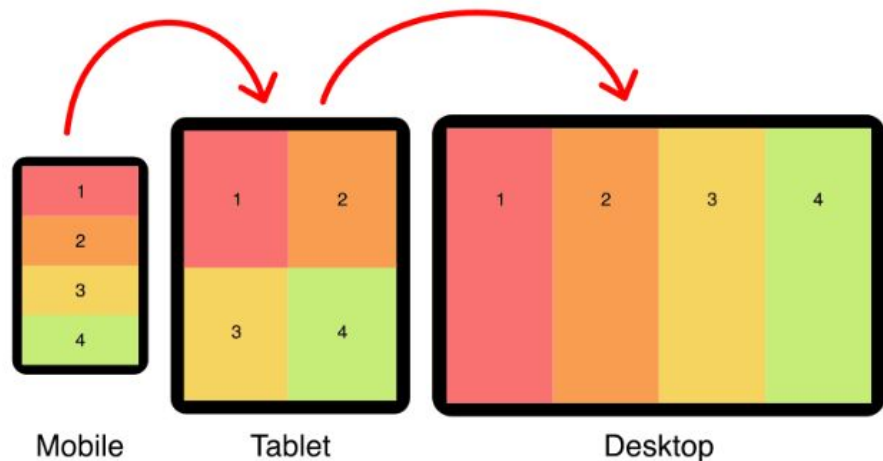
- sm: 640px @media (min-width: 640px)
- md: 768px @media (min-width: 768px)
- lg: 1024px @media (min-width: 1024px)
- xl: 1280px @media (min-width: 1280px)
- **Unprefixed** utilities target mobile (all screens).

Responsive Layouts

From Stack to Grid

A common pattern: stacking items vertically on mobile, then switching to a multi-column grid on tablets and desktops.

This single line of code handles three different device layouts without a single line of custom CSS.



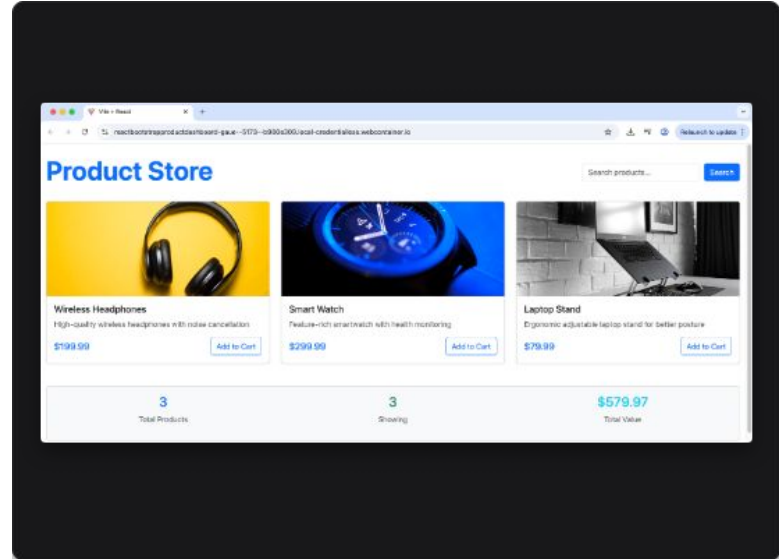
Component Libraries



The Classic: React Bootstrap

Rapid Prototyping

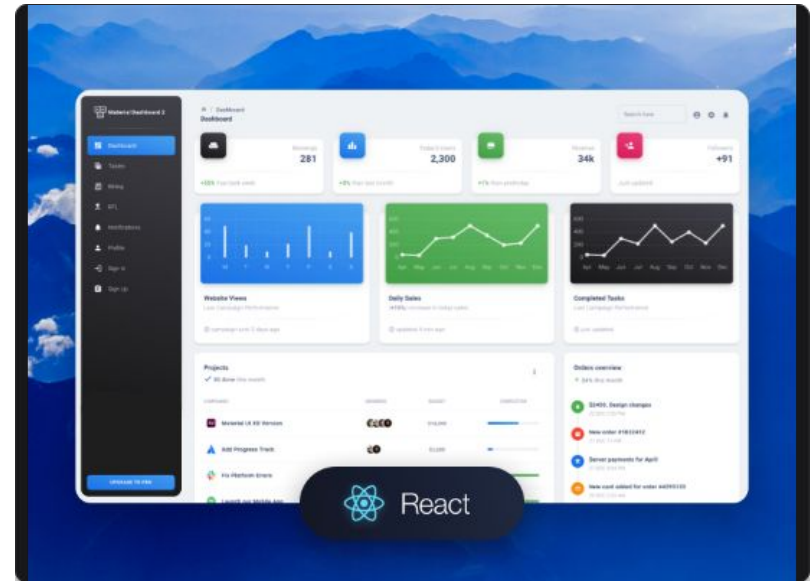
- **Legacy Strength:** React Bootstrap rebuilds Bootstrap's JS components (Modals, Tooltips) as React components.
- **Pros:** Immediate familiarity, rock-solid grid system, thousands of themes available.
- **Cons:** "Bootstrap Look" is hard to shake, heavier bundle sizes, and harder to customize deeply without !important.



The Enterprise Standard: Material UI

System Completeness

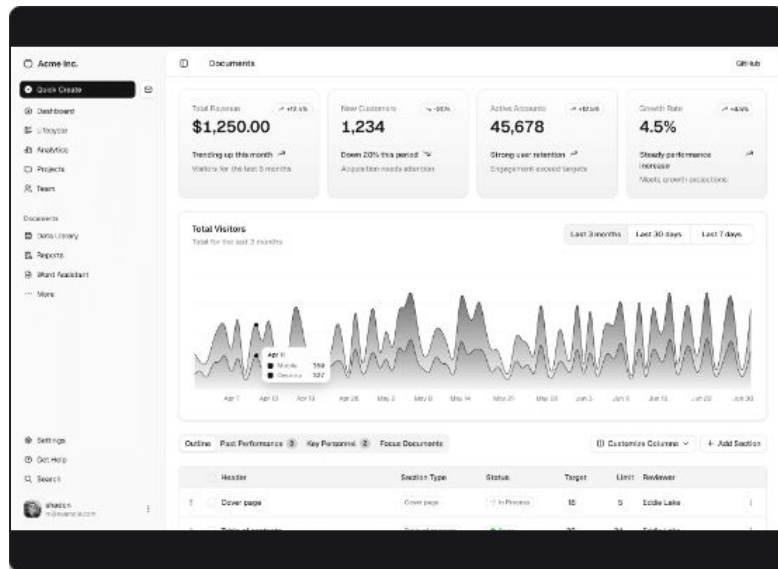
- **The Heavyweight:** MUI (Material UI) implements Google's Material Design system perfectly.
- **Pros:** Incredible accessibility, massive component suite (DataGrid, DatePicker), enterprise-ready.
- **Cons:** Can feel rigid ("Google look"), significant bundle size, and complex theming API (historically).



What is Shadcn UI?

It's Not a Component Library

- Most libraries (MUI, Bootstrap) exist as a dependency in `node_modules`.
- **Shadcn UI** is a collection of re-usable components that you copy and paste into your apps.
- **Ownership:** The code lives in *your* project (e.g., `components/ui/button.tsx`). You have full control to customize it.



The Architecture

Radix + Tailwind

Shadcn UI is built on top of **Radix UI** (headless primitives) and styled with **Tailwind CSS**.

Radix handles the hard stuff: accessibility (ARIA), keyboard navigation, and focus management.

Tailwind handles the styling, making it easy to theme and adapt to your brand.

Radix Primitives

